

PISSIR - Relazione finale

Connected Games Platform - PlayConnect

Piattaforma per collegare giochi fisici tradizionali a Internet.

Anno accademico 2025/2026 Docente: Alberto Testa

Studenti / gruppo:

- Tchamba Rayan - 20054911
- Nguetchouo Andre-Junior - 20056497
- Beack Maurice - 20054532

Indice

1. Introduzione
2. Specifiche funzionali
3. Analisi tecnologica
4. Scelta dell'approccio
5. Architettura
6. Implementazione
7. Coerenza con il laboratorio
8. Validazione
9. Sicurezza e scelte didattiche
10. Conclusione
11. Appendici

1. Introduzione

PlayConnect è una piattaforma che collega a Internet giochi fisici tradizionali, per esempio calciobalilla, freccette, bocce e Monopoli. I giochi continuano a essere usati normalmente, ma alcuni sensori rilevano eventi importanti. Un dispositivo edge installato nel locale riceve questi eventi e li invia al server tramite MQTT.

Il server salva partite, punteggi e statistiche. Gli utenti possono vedere i giochi disponibili, le proprie partite, i tornei e le classifiche. Gli amministratori possono configurare locali, giochi, sensori e dispositivi.

Il progetto è stato realizzato con codice volutamente semplice, diviso in piccoli moduli facili da spiegare durante l'esame.

2. Specifiche funzionali

2.1 Utenti

Il sistema gestisce quattro ruoli.

Giocatore

- vede i giochi online;
- consulta le proprie partite;
- consulta statistiche e classifiche;
- partecipa a tornei individuali o come membro di una squadra.

Amministratore del locale

- gestisce i giochi del proprio locale;
- crea giocatori del proprio locale;
- configura dispositivi edge, sensori e attuatori;
- avvia partite individuali o a squadre;
- controlla partite e statistiche locali.

Amministratore del gioco

- definisce nuovi tipi di gioco;
- specifica gli eventi di inizio, punteggio e fine;
- crea modelli di sensore associati ai tipi di gioco;
- controlla le configurazioni di sensori e attuatori installati.

Amministratore della piattaforma

- gestisce utenti e locali;
- crea amministratori locali e amministratori del gioco;
- crea tornei;
- seleziona i locali coinvolti nei tornei;
- controlla statistiche globali.

2.2 Giochi, sensori e attuatori

Ogni gioco appartiene a un locale e a un tipo di gioco. Un tipo di gioco contiene regole semplici: evento di inizio, eventi di punteggio, evento di fine, eventuale limite di punteggio e possibilità di giocare a squadre.

I sensori possono essere fisici oppure simulati. Ogni sensore ha un evento associato e un topic MQTT.

Sono presenti attuatori simulati come display del punteggio, LED ed eventuale buzzer. Lo stato dell'attuatore viene aggiornato quando la partita inizia, quando cambia il punteggio e quando termina.

2.3 Dispositivo edge

Il dispositivo edge:

- coordina i giochi del locale;
- riceve o simula gli eventi dei sensori;
- pubblica gli eventi su MQTT;
- mantiene una coda JSON quando la connessione non è disponibile;
- sincronizza la coda quando torna online;
- riceve comandi per gli attuatori;
- offre una piccola interfaccia locale sulla porta 8090.

2.4 Partite e tornei

Una partita può essere individuale, con due giocatori registrati, oppure a squadre, con due squadre registrate. Il server registra gioco, locale, partecipanti, punteggio, vincitore, stato, data di inizio e fine, e lista ordinata degli eventi.

Un torneo riguarda un solo tipo di gioco, può coinvolgere uno o più locali, può essere individuale oppure a squadre, contiene partite divise per turno e produce una classifica con 3 punti per vittoria e 1 punto per pareggio.

3. Analisi tecnologica

Sono state considerate più tecnologie di comunicazione.

REST diretto dal dispositivo è semplice da capire e da provare con Postman, ma il dispositivo deve conoscere direttamente il server e la gestione delle disconnessioni è meno naturale.

WebSocket permette comunicazione bidirezionale veloce, ma richiede una gestione più complessa della riconnessione ed è meno adatto a tanti piccoli dispositivi IoT.

Server-Sent Events è utile per aggiornare una pagina live, ma la comunicazione è principalmente server-verso-client e non sostituisce il canale dei sensori.

MQTT con message broker usa topic e un broker centrale. Il dispositivo pubblica un messaggio senza conoscere direttamente chi lo userà. È leggero, adatto ai sensori, disaccoppia edge e server, supporta QoS e riconnessione.

Il dispositivo edge può trovarsi dietro un router privato. Non è necessario aprire porte in ingresso, perché è il dispositivo a creare una connessione in uscita verso il broker MQTT.

Gli eventi sono piccoli messaggi JSON, normalmente inferiori a 1 KB. Una partita produce pochi eventi al secondo. Il progetto usa MQTT QoS 1, quindi il messaggio viene consegnato almeno una volta. Per evitare che un evento ripetuto aumenti due volte il punteggio, ogni evento contiene un `event_uuid` unico.

4. Scelta dell'approccio

È stato scelto MQTT per la comunicazione tra edge e server.

La scelta è adatta perché:

- i sensori inviano messaggi piccoli;
- il dispositivo può perdere temporaneamente la connessione;
- il broker separa chi produce gli eventi da chi li elabora;
- è possibile aggiungere altri servizi che ascoltano gli stessi topic;
- la struttura dei topic identifica locale, gioco e partita.

REST viene comunque usato tra interfaccia web e server. In questo modo ogni tecnologia viene usata per il compito più semplice:

- REST: gestione di utenti, giochi, tornei e consultazione dati;
- MQTT: eventi dei sensori e comandi degli attuatori.

I laboratori mostrano spesso esempi in Java, Spark e JDBC. Nel progetto è stata scelta una implementazione Node.js con Express e mysql2, perché il testo del progetto richiede un sistema funzionante e documentato ma non impone un linguaggio unico. La scelta rimane coerente con i concetti del corso: Express realizza server REST con risorse URI e metodi HTTP, mysql2 svolge il ruolo di driver verso MySQL, il runtime Node gestisce richieste concorrenti ed eventi tramite modello asincrono, e i container Docker rendono ripetibile l'esecuzione dei servizi.

5. Architettura

Il sistema è organizzato con microservizi e container Docker.

Componente	Porta	Input	Output	Business logic
Frontend	8080	Azioni utente	Pagine HTML	Interfaccia per ruoli
API Gateway	3000	Richieste /api	Risposta del servizio	Instradamento verso il servizio corretto
Catalog Service	3001	REST	JSON/MySQL	Utenti, locali, tipi di gioco, giochi, edge, sensori, attuatori
Match Service	3002	REST + MQTT	JSON + comandi MQTT	Partite, punteggi, eventi, deduplicazione
Tournament Service	3003	REST	JSON/MySQL	Squadre, tornei, calendario, statistiche e classifica

Componente	Porta	Input	Output	Business logic
Edge Service	8090	REST locale + sensori	MQTT	Simulazione sensori, coda offline, attuatori locali
Mosquitto	1883	Messaggi MQTT	Messaggi MQTT	Broker dei messaggi
MySQL	3306	Query dei servizi	Dati persistenti	Database centrale

I tre servizi applicativi sono eseguiti in container separati. Ogni servizio espone solo le API del proprio ambito. Il gateway nasconde questa divisione al frontend.

Il database è condiviso per mantenere il progetto didattico semplice. Solo i componenti server accedono direttamente a MySQL; frontend ed edge non accedono mai al database.

Nel progetto sono presenti più attività concorrenti: Express riceve richieste REST, il Match Service ascolta i topic MQTT, il client MQTT tenta la riconnessione, l'Edge Service invia heartbeat ogni 5 secondi, l'Edge Service sincronizza la coda offline, il frontend aggiorna pagine live con polling e una simulazione partita genera eventi con timer.

6. Implementazione

I controller ricevono la richiesta, controllano ruolo e dati e chiamano i servizi. Esempi principali:

- `gameTypeController.js` gestisce tipi di gioco e modelli sensore;
- `deviceController.js` gestisce edge, sensori e attuatori;
- `matchController.js` avvia partite e controlla gli accessi;
- `tournamentController.js` gestisce locali, squadre e calendario del torneo.

I service contengono la logica riutilizzabile:

- `matchEventService.js` aggiorna punteggi e salva eventi;
- `actuatorService.js` aggiorna display e LED;
- `tournamentService.js` costruisce classifica e collegamenti.

Le utility contengono regole semplici e testabili: autorizzazioni, validazione dei dati, calcolo della classifica e controllo compatibilità partita-torneo.

Sono presenti pagine diverse per ogni ruolo: piattaforma, locale, gioco e giocatore. L'interfaccia edge locale mostra stato MQTT, coda offline, ultimo evento, attuatori ricevuti e pulsanti per simulare online/offline.

7. Coerenza con il laboratorio

Il progetto riprende direttamente i temi principali dei laboratori.

Tema di laboratorio	Applicazione nel progetto
REST, HTTP e progettazione URI	API documentata in <code>docs/openapi.yaml</code> , gateway <code>/api</code> , risorse per utenti, giochi, edge, sensori, partite, tornei e statistiche
Server concorrenti e attività asincrone	Richieste Express, listener MQTT, heartbeat edge, sincronizzazione offline e polling live eseguiti senza bloccare il server
Publish/Subscribe e message broker	Broker Mosquitto, topic per eventi dei sensori, topic per comandi agli attuatori, QoS 1 e deduplicazione con UUID
IoT, sensori e attuatori	Edge locale, sensori simulati configurabili, coda offline, display/LED/buzzer simulati

Tema di laboratorio	Applicazione nel progetto
Microservices architecture style	API Gateway, Catalog Service, Match Service e Tournament Service separati in container Docker
Accesso a database	MySQL persistente, schema SQL, driver applicativo mysql2, query parametrizzate e bootstrap dello schema
OAuth2, Keycloak e TLS	Studiati come evoluzione produttiva; nella demo didattica si usano login locale, hash PBKDF2, header X-User-Id e HTTP locale

Non sono emersi nei materiali di laboratorio divieti incompatibili con l'architettura realizzata. Le tecnologie Java/Spark/JDBC sono usate nei laboratori come strumenti didattici; nel progetto sono state sostituite da strumenti equivalenti mantenendo gli stessi principi di rete, persistenza, concorrenza e comunicazione event-driven.

8. Validazione

I test automatici verificano:

- permessi dei ruoli;
- calcolo del vincitore;
- classifica torneo;
- ruolo amministratore gioco;
- validazione dei tipi di gioco;
- torneo con più locali;
- partite individuali e a squadre;
- configurazione sensori;
- compatibilità tra torneo e partita.

Comandi:

```
cd backend
npm test
npm run check
cd ..
node scripts/validate-project.js
```

Con tutti i container avviati si esegue:

```
node scripts/integration-test.js
```

Il test di integrazione controlla API Gateway, tre microservizi, Edge Service, login dei quattro ruoli, Catalog Service, Match Service e Tournament Service.

La versione aggiornata del test di integrazione verifica anche uno scenario end-to-end reale: creazione di un nuovo gioco di test, creazione di un attuttore, avvio di una partita, invio di un evento MQTT online, simulazione offline dell'edge, salvataggio nella coda JSON, ritorno online, sincronizzazione dell'evento, deduplicazione tramite UUID e aggiornamento finale dell'attuttore.

La demo manuale mostra login, creazione tipo di gioco, configurazione edge/sensori/attuatori, avvio partita, simulazione MQTT, aggiornamento punteggio, funzionamento offline, sincronizzazione, torneo multi-locale e classifica.

9. Sicurezza e scelte didattiche

Per rendere il progetto facile da eseguire e spiegare:

- le password demo sono salvate nel database come hash PBKDF2;
- il bootstrap migra automaticamente eventuali password legacy in chiaro verso password_hash;

- l'autenticazione REST demo usa ancora l'header X-User-Id dopo il login;
- i microservizi condividono lo stesso database;
- non è configurato HTTPS.

Queste scelte restano didattiche. In produzione si userebbero JWT o sessioni firmate, HTTPS, segreti esterni, rotazione delle credenziali e database separati o ownership più rigida per ogni microservizio.

10. Conclusione

Il progetto soddisfa le funzionalità richieste: giochi connessi, quattro ruoli, sensori, attuatori, edge offline, MQTT, REST, microservizi, partite individuali e a squadre, tornei multi-locale, classifiche, statistiche, test e demo.

L'architettura rimane semplice: ogni modulo ha un compito chiaro e il flusso principale può essere spiegato seguendo un evento dal sensore fino al database e al display.

Appendice A - Casi d'uso testuali

I casi d'uso testuali completi sono disponibili in docs/13-casi-uso-testuali.pdf.

I casi principali sono:

- login;
- definizione di un tipo di gioco;
- installazione di un gioco nel locale;
- configurazione edge, sensore e attuttore;
- avvio partita individuale;
- avvio partita a squadre;
- ricezione eventi MQTT;
- funzionamento offline;
- creazione torneo multi-locale;
- collegamento partita al calendario.

Appendice B - Matrice dei requisiti

La matrice requisiti completa è disponibile in docs/16-matrice-requisiti.pdf.

Il progetto copre giocatori, amministratori, tipi di gioco, sensori, edge locale, offline, MQTT, attuatori, REST, microservizi, partite individuali, partite a squadre, tornei multi-locale, calendario, classifica, statistiche, diagrammi, OpenAPI e test.

Appendice C - Procedura demo

Preparazione:

```
docker compose down -v
docker compose up --build -d
```

Aprire:

- piattaforma: <http://localhost:8080>
- edge locale: <http://localhost:8090>
- health API: <http://localhost:3000/api/health>

Sequenza consigliata:

1. Login gameadmin / game123.
2. Mostrare tipi di gioco e modelli sensore.

3. Login `localadmin` / `local123`.
4. Mostrare edge, sensori e display.
5. Mostrare squadre.
6. Avviare partita individuale Mario-Luigi.
7. Premere simulazione MQTT.
8. Mostrare punteggio ed eventi ricevuti.
9. Mostrare stato attuatore su `localhost:8090`.
10. Simulare offline e inviare un evento manuale.
11. Tornare online e verificare che la coda si svuoti.
12. Login `platform` / `platform123`.
13. Mostrare utenti con i quattro ruoli.
14. Mostrare tornei, locali coinvolti, calendario e classifica.
15. Concludere mostrando test e matrice requisiti.

Frase semplice per spiegare il flusso:

> Il sensore genera un evento, l'edge lo mette su un topic MQTT, il Match Service lo controlla e lo salva, poi aggiorna il punteggio e invia lo stato al display. Se Internet manca, l'edge conserva l'evento e lo invia dopo.